Task 3 — Automated File Organizer & Cleanup Utility

Task overview and instructions

Features

• Sorts files by extension into `Documents`, `Images`, `Videos`, `Archives`, and `Code`

• Creates category directories automatically

• Uses `pathlib`, `os.scandir`, and `shutil.move`

• Handles permissions and other filesystem errors without crashing the whole run

• Resolves duplicate destination names by appending a number, never overwriting

• Dry-run mode is enabled by using `--dry-run`

Run safely

Preview first:

python organizer.py --source "C:/Users/YourName/Downloads" --dry-run

After reviewing the preview, perform moves:

python organizer.py --source "C:/Users/YourName/Downloads"

Important: Replace the example path with your actual folder. Use a test folder first. The script only processes files directly in the selected folder, not nested subfolders. It never deletes files.

Python source code: organizer.py

```
"""Organize files into category folders; supports dry-run and collision-safe moves."""
import argparse
import os
import shutil
from pathlib import Path

EXTENSION_CATEGORIES = {
    ".pdf": "Documents", ".doc": "Documents", ".docx": "Documents",
    ".txt": "Documents", ".rtf": "Documents", ".odt": "Documents",
    ".jpg": "Images", ".jpeg": "Images", ".png": "Images",
    ".gif": "Images", ".bmp": "Images", ".svg": "Images", ".webp": "Images",
    ".mp4": "Videos", ".mov": "Videos", ".avi": "Videos",
    ".mkv": "Videos", ".webm": "Videos",
    ".zip": "Archives", ".rar": "Archives", ".7z": "Archives",
    ".tar": "Archives", ".gz": "Archives",
    ".py": "Code", ".js": "Code", ".ts": "Code", ".html": "Code",
    ".css": "Code", ".java": "Code", ".c": "Code", ".cpp": "Code",
    ".h": "Code", ".cs": "Code", ".json": "Code", ".xml": "Code",
    ".sql": "Code", ".ipynb": "Code",
}


def unique_destination(path: Path) -> Path:
    """Return a non-existing path by appending a numeric suffix if needed."""
    if not path.exists():
        return path
    index = 1
    while True:
        candidate = path.with_name(f"{path.stem}_{index}{path.suffix}")
        if not candidate.exists():
            return candidate
        index += 1


def organize(source: Path, dry_run: bool = False) -> tuple[int, int]:
    if not source.exists() or not source.is_dir():
        raise ValueError(f"Source folder does not exist or is not a directory: {source}")

    moved = 0
    skipped = 0
```

```
    # scandir is used as requested; errors are caught per entry.
    with os.scandir(source) as entries:
        for entry in entries:
            try:
                if not entry.is_file(follow_symlinks=False):
                    continue
                src = Path(entry.path)
                category = EXTENSION_CATEGORIES.get(src.suffix.lower())
                if category is None:
                    print(f"Skipped (unknown extension): {src.name}")
                    skipped += 1
                    continue

                folder = source / category
                target = unique_destination(folder / src.name)
                if dry_run:
                    print(f"[DRY RUN] {src.name} -> {category}/{target.name}")
                    moved += 1
                    continue

                folder.mkdir(parents=True, exist_ok=True)
                shutil.move(str(src), str(target))
                print(f"Moved: {src.name} -> {category}/{target.name}")
                moved += 1
            except PermissionError as exc:
                print(f"Permission denied for {entry.name}: {exc}")
                skipped += 1
            except (OSError, shutil.Error) as exc:
                print(f"Could not process {entry.name}: {exc}")
                skipped += 1

    print("\n=== ORGANIZER SUMMARY ===")
    print(f"Files {'planned' if dry_run else 'moved'}: {moved}")
    print(f"Files skipped: {skipped}")
    if dry_run:
        print("Dry-run only: no files were moved. Run without --dry-run to apply changes.")
    return moved, skipped


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source", required=True, type=Path, help="Folder to organize")
    parser.add_argument("--dry-run", action="store_true", help="Preview moves without changing
     files")
    args = parser.parse_args()
    try:
        organize(args.source, args.dry_run)
    except (OSError, ValueError) as exc:
        parser.error(str(exc))


if __name__ == "__main__":
    main()
```